OLS Regularization Techniques

LASSO, Ridge Regression, and Elastic Net

GDP Nowcasting Workshop

Central Bank of Trinidad and Tobago

2026

Part I

Why Regularize?

The Failures of OLS

OLS is the workhorse of econometrics — but it breaks down in three situations that arise constantly in nowcasting.

Multicollinearity

Predictors move together (natural gas production & LNG output). OLS cannot separate their contributions — coefficients explode and flip signs.

Symptom: VIF $\gg 1$; signs contradict theory.

High Dimensionality ($p \geq n$)

Trinidad and Tobago has $\approx$60 quarterly observations but 600+ candidate indicators.

When $p \geq n$, OLS is **undefined** — more unknowns than equations. Any solution fits perfectly with $R^2 = 1$, but generalises to nothing.

Overfitting

OLS minimises *training* error. A model that memorises 2012–2022 noise predicts 2023 GDP poorly.

Key insight: what matters is **test-set error** — performance on data the model has never seen.

Nowcasting context

In the Trinidad and Tobago quarterly nowcast: $n \approx 60$ obs., $p \approx 600+$ candidate indicators. This is the regime where OLS breaks and regularization is **necessary**.

The Core Trade-off: Bias vs. Variance

What do these words mean?

Imagine re-estimating your nowcast model on 100 different quarterly samples drawn from the same economy.

- **Variance** — how much the coefficient estimates *scatter* across those 100 runs. Under multicollinearity, OLS estimates jump wildly from one sample to the next.
- **Bias** — whether the *average* estimate across runs points at the right answer. OLS is unbiased by the Gauss–Markov theorem.

The punchline: OLS has Bias = 0 but Variance can be enormous. Regularization accepts a small Bias to collapse Variance ⇒ **lower test-set error**.

The rifle analogy



A tight cluster slightly left of centre is more *useful* than one scattered all over the target, even though it is technically "biased".

The Penalized Regression Framework

All three methods minimise the same objective: fit the data, but **pay a penalty** for large coefficients.

$$\hat{\beta} = \arg\min_{\beta} \Big[\underbrace{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}_{\text{SSR}} + \underbrace{\lambda \cdot J(\beta)}_{\text{penalise size}} \Big]$$

where:

- $J(\beta)$ — penalty functional; choice of J defines the method
- $\lambda \geq 0$ — penalty strength: $\lambda{=}0$ recovers OLS; $\lambda{\to}\infty$ forces all $\hat{\beta}_j \to 0$

Method	Penalty $J(\beta)$	What it does
LASSO	$\lvert\beta_1\rvert + \lvert\beta_2\rvert + \cdots + \lvert\beta_p\rvert$	Drives some $\hat{\beta}_j$ to **exactly zero**
Ridge	$\beta_1^2 + \beta_2^2 + \cdots + \beta_p^2$	Shrinks all $\hat{\beta}_j$; none reach zero
Elastic Net	$\alpha \sum \lvert\beta_j\rvert + (1{-}\alpha) \sum \beta_j^2$	Selection *and* stability under collinearity

Important: standardise all predictors to zero mean and unit variance before fitting. The intercept β_0 is never penalised.

Part II

Ridge Regression

Ridge Regression — Objective Function

L2 Regularization Hoerl & Kennard (1970). Technometrics, 12(1).

Ridge adds the sum of *squared* coefficients to the SSR:

$$\hat{\beta}^{\text{Ridge}} = \arg\min_{\beta} \Big[\underbrace{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}_{\text{SSR}} + \underbrace{\alpha \left(\beta_1^2 + \beta_2^2 + \cdots + \beta_p^2 \right)}_{\text{L2 penalty}} \Big]$$

where:

- α — penalty strength (`alpha` in sklearn; $= \lambda$ in textbooks)
- β_j^2 — squared coefficient; differentiable everywhere, *including at zero*

Key properties

- **Closed-form solution** — always exists, even when $p \geq n$
- Shrinks *all* coefficients toward zero; none reach exactly zero
- Directly fixes multicollinearity

Hoerl–Kennard Theorem (1970)

There *always* exists $\alpha^* > 0$ such that Ridge has **lower test error than OLS**.

Accepting a small bias collapses variance enough to win on net.

Ridge — Shrinkage Factor & `sklearn`

One-predictor intuition

With a single predictor x, the Ridge estimate is:

$$\hat{\beta}^{\text{Ridge}} = \underbrace{\frac{\sum_i x_i^2}{\sum_i x_i^2 + \alpha}}_{\text{shrinkage factor} \in (0,1)} \times \hat{\beta}^{\text{OLS}}$$

Ridge **multiplies** every OLS estimate by a number strictly between 0 and 1. Multiplication can never produce exactly zero.

α	Shrinkage factor
0	1.00 (= OLS)
1	0.83
10	0.50
100	0.09
∞	0.00

`sklearn` **API**

```
from sklearn.linear_model import (
    Ridge, RidgeCV)

# Fixed penalty
model = Ridge(
    alpha=1.0,
    solver='auto'    # exact algebra
)
model.fit(X_train, y_train)
print(model.coef_)

# Auto-select alpha via CV
cv_model = RidgeCV(
    alphas=[0.1, 1.0, 10.0],
    cv=5
).fit(X, y)
print(cv_model.alpha_)
```

Note: Ridge has an exact algebraic solution — no `max_iter` needed.

Part III

LASSO

LASSO — Objective Function

L1 Regularization Tibshirani, R. (1996). JRSS-B, 58(1), 267–288.

LASSO replaces squared coefficients with their *absolute values*:

$$\hat{\beta}^{\text{LASSO}} = \arg\min_{\beta} \Bigg[\underbrace{\frac{1}{2n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}_{\text{scaled SSR}} + \underbrace{\alpha \big(|\beta_1| + |\beta_2| + \cdots + |\beta_p| \big)}_{\text{L1 penalty}} \Bigg]$$

where:

- α — regularization strength (`alpha` in sklearn)
- $|\beta_j|$ — absolute value; **not smooth at zero** — this is exactly what produces exact zeros

Why is L1 different from L2?

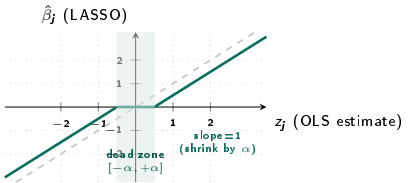
The squared penalty β_j^2 has derivative $2\beta_j$ which approaches *zero* as $\beta_j \to 0$ — the penalty gives up before reaching zero.

The absolute-value penalty $|\beta_j|$ has derivative ± 1 everywhere except at zero — it pushes with **constant force** all the way to zero and holds the coefficient there.

LASSO — Soft-Thresholding

Coordinate descent solves LASSO by updating one β_j at a time. The update for each β_j is the **soft-thresholding function** — a piecewise rule with three explicit cases:

Visual: the piecewise function



Dashed = OLS. LASSO subtracts/adds α; zeros $|z_j| \leq \alpha$.

Piecewise definition

$$\hat{\beta}_j = \begin{cases} z_j - \alpha & \text{if } z_j > \alpha \\ 0 & \text{if } |z_j| \leq \alpha \\ z_j + \alpha & \text{if } z_j < -\alpha \end{cases}$$

where:

- z_j — OLS estimate (others fixed)
- α — penalty (`alpha`)
- Dead zone: $|z_j| \leq \alpha$ maps to 0

Worked example, $\alpha = 0.5$:

	z_j	Case	$\hat{\beta}_j$
Gas prod.	$+3.1$	$z_j > \alpha$	2.6
Rainfall	$+0.4$	dead zone	0.0
FX rate	-1.7	$z_j < -\alpha$	-1.2

LASSO — sklearn Implementation

```
from sklearn.linear_model import (
    Lasso, LassoCV)

# Fixed penalty
model = Lasso(
    alpha=1.0,          # lambda
    max_iter=1000,      # coord. descent
    tol=1e-4,
    selection='cyclic'  # or 'random'
)
model.fit(X_train, y_train)
print(model.coef_)      # many zeros
print(model.intercept_)

# Auto-select alpha via CV
cv_model = LassoCV(
    cv=5,
    max_iter=5000
).fit(X, y)
print(cv_model.alpha_)
```

Key parameters

- `alpha` — the λ penalty. Larger $\Rightarrow$ more zeros. Always tune via CV.
- `max_iter` — coordinate descent steps. Increase if you see a convergence warning.
- `selection='cyclic'` — cycles through predictors in order; `'random'` can be faster for very large p.
- `LassoCV` — searches a grid of α values using k-fold CV automatically.

After fitting, `model.coef_` will contain many exact zeros — a natural feature selection step for indicator screening.

Part IV

Elastic Net

Why LASSO Is Not Always Enough

LASSO has two specific failure modes that matter in macroeconomic forecasting:

Failure 1: Arbitrary group selection

When indicators are correlated (gas production, LNG exports, ammonia and methanol output — all highly collinear), LASSO picks **one arbitrarily** and zeros the rest.

The selected indicator may differ between runs if training data changes slightly. This is economically uninterpretable and statistically unstable.

Failure 2: Saturation at n variables

When $p > n$, LASSO can select **at most n variables** before it saturates.

With 60 quarterly observations and 600+ candidate indicators, LASSO cannot use more than 60 regardless of how many are genuinely informative. This is a hard mathematical limit, not a tuning problem.

The fix: Elastic Net

Elastic Net adds a Ridge-type (L2) component alongside the LASSO component.

Grouping effect (Zou & Hastie, 2005): if two predictors are perfectly correlated, Elastic Net assigns them *equal* coefficients — both in or both out.

No saturation: Elastic Net can select all p variables because the L2 term resolves the ambiguity that causes LASSO to stall.

Elastic Net — Objective Function

L1+L2 Hybrid Zou & Hastie (2005). JRSS-B, 67(2), 301–320.

Elastic Net combines both penalties. The **sklearn objective** is:

$$\hat{\beta}^{\text{EN}} = \arg\min_{\beta} \left[\frac{1}{2n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \underbrace{\alpha \cdot \ell_1 \sum_j |\beta_j|}_{\text{LASSO part}} + \underbrace{\frac{\alpha(1-\ell_1)}{2} \sum_j \beta_j^2}_{\text{Ridge part}} \right]$$

where:

- `alpha` (α) — overall penalty strength
- `l1_ratio` ($\ell_1 \in [0, 1]$) — mix: ℓ_1=1 is pure LASSO; ℓ_1=0 is pure Ridge
- **Notation warning:** sklearn's `alpha` $= \lambda$ in the literature; sklearn's `l1_ratio` $= \alpha$ in Zou & Hastie — names are swapped vs `glmnet` (R)

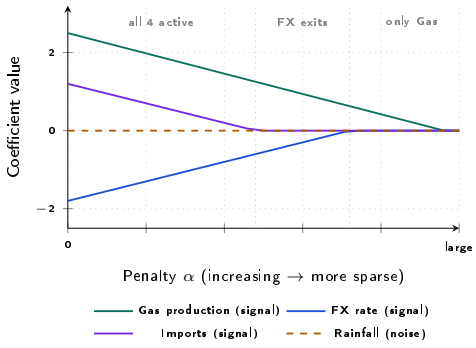
`l1_ratio`	Equivalent to	Behaviour
1.0	pure LASSO	exact zeros; may pick arbitrarily
0.5	Elastic Net	zeros + grouping of correlated vars
0.0	pure Ridge	no zeros; stable under collinearity

Part V

Regularization Paths & Tuning

Regularization Paths

A **regularization path** shows how every coefficient changes as α varies from 0 (OLS) to ∞ (all zero). It reveals *which* variables are informative and in *what order* they enter the model.



How to read a LASSO path

- At $\alpha = 0$: all coefficients equal OLS values
- As α increases: coefficients shrink and hit **zero one at a time**
- The **order of exit** is informative — the last variable to survive is the single most predictive one
- Noise variables (true $\beta = 0$) are zeroed early; signal variables survive longer

```
from sklearn.linear_model import lasso_
    path
alphas, coefs, _ = lasso_path(X, y)
# coefs shape: (p, n_alphas)
```

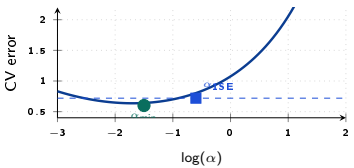
Tuning α: Cross-Validation

Why we need CV

We cannot pick α by looking at training error — that always prefers $\alpha = 0$ (OLS). We need an estimate of performance on **unseen data**.

k-fold procedure

1. Split data into k folds (typically 5 or 10)
2. For each α, train on $k-1$ folds, evaluate on the held-out fold
3. Average errors across folds
4. Pick the α with lowest average error



The 1-SE rule

- α_{min} (circle): minimises CV error
- $\alpha_{1\text{SE}}$ (square): the **largest** α within 1 SE of the minimum
- Produces a **more parsimonious model** (fewer predictors) at negligible accuracy cost
- Prefer $\alpha_{1\text{SE}}$ when interpretability matters — e.g. identifying which indicators actually drive GDP

sklearn one-liners:

```
from sklearn.linear_model import (
    LassoCV, RidgeCV, ElasticNetCV)
LassoCV(cv=5).fit(X,y).alpha_
RidgeCV(cv=5).fit(X,y).alpha_
ElasticNetCV(l1_ratio=[.5,.7,.9,1.],
    cv=5).fit(X,y).alpha_
```

Part VI

Method Comparison

Comparison & Decision Guide

Property	LASSO	Ridge	Elastic Net
Exact zeros	Yes	No	Yes (`l1_ratio` > 0)
Closed form	No (iterative)	Yes	No (iterative)
$p > n$	Up to n vars	All p vars	All p vars
Correlated features	Picks one	Shrinks equally	Groups together
sklearn CV	`LassoCV`	`RidgeCV`	`ElasticNetCV`
When to use	Sparse signal; need selection	High collinearity; keep all	Safe default for nowcasting